

- Introduction

- 1. Setting Up the Built-In Logger
- 2. Implementing a Custom Logger
- 3. Integrating an External Logger

- Conclusion

How to Setup Inbuilt, Custom, and External Loggers in Your NestJS Project

A Step-by-Step Guide on How to Set Up Your NestJS Logger

Introduction

Logging is an essential part of any application, helping developers track the behavior of their code, identify errors, and gain insights into performance issues. NestJS provides robust tools for logging, including built-in loggers, custom loggers, and the ability to integrate external logging packages. In this tutorial, we'll walk through how to set up each of these logging systems in your NestJS project.

1. Setting Up the Built-In Logger

The built-in logger in NestJS is provided by the `Logger` class in the `@nestjs/common` package. This logger is useful for basic logging tasks, such as recording errors, warnings, and general application behavior. It's simple to set up and can be customized to meet specific needs.

The built-in logger is perfect for quick setups where you need to log essential application events without too much complexity. It's a great starting point for most projects and can be extended if necessary.

Step-by-Step Guide

Step 1: Enable the Inbuilt Logger

By default, the inbuilt logger is enabled in a NestJS project. However, you can customize its behavior.

```
> _ Terminal Copy

import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    logger: ['error', 'warn', 'log'], // Customize log levels
  });
  await app.listen(3000);
}
bootstrap();
```

Step 2: Disable the Logger

If you want to disable logging entirely, you can set the `logger` property to `false`.

```
> _ Index.js Copy

const app = await NestFactory.create(AppModule, {
  logger: false, // Disable logging
});
```

Step 3: Change Log Levels

You can specify which log levels to display by passing an array of log levels to the `logger` property.

For example:

```
> _ Index.js Copy

const app = await NestFactory.create(AppModule, {
  logger: ['error', 'warn'], // Only show errors and warnings
});
```

Step 4: Disable Colors in Logs

If you want to remove colors from log messages, set the `NO_COLOR` environment variable.

```
> _ Index.js Copy

NO_COLOR=true
This is helpful if you're logging into a file or a system that doesn't handle colored text well.
```

2. Implementing a Custom Logger

For more complex logging requirements, you might want to create a custom logger. This allows you to tailor the logging behavior to your specific needs, whether that's adding additional log levels, formatting, or integrating with other parts of your application.

A custom logger is beneficial when you need more control over logging. For instance, if you want to integrate logging with a monitoring service, add custom log formats, or extend the functionality of the default logger.

Step-by-Step Guide

Step 1: Create a Custom Logger

Start by implementing the `LoggerService` interface.

```
> _ Terminal Copy

import { LoggerService, Injectable } from '@nestjs/common';

@Injectable()
export class MyLogger implements LoggerService {
  log(message: string) {
    console.log('Log:', message);
  }

  error(message: string, trace: string) {
    console.error('Error:', message, trace);
  }

  warn(message: string) {
    console.warn('Warn:', message);
  }

  debug(message: string) {
    console.debug('Debug:', message);
  }

  verbose(message: string) {
    console.log('Verbose:', message);
  }
}
```

Step 2: Register the Custom Logger

You'll need to register your custom logger in a module.

```
> _ Index.js Copy

import { Module } from '@nestjs/common';
import { MyLogger } from './my-logger.service';

@Module({
  providers: [MyLogger],
  exports: [MyLogger],
})
export class LoggerModule {}
```

Step 3: Use the Custom Logger in the Application

Next, configure your NestJS application to use the custom logger.

For example:

```
> _ Index.js Copy

const app = await NestFactory.create(AppModule, {
  logger: new MyLogger(),
});
await app.listen(3000);
```

Step 4: Inject the Custom Logger

You can inject your custom logger into other services or controllers using dependency injection.

```
> _ Index.js Copy

import { Injectable } from '@nestjs/common';
import { MyLogger } from './my-logger.service';

@Injectable()
export class MyService {
  constructor(private readonly logger: MyLogger) {}

  doSomething() {
    this.logger.log('Doing something...');
  }
}
```

3. Integrating an External Logger

In some cases, you might need advanced logging features such as centralized logging, log rotation, or custom log formats. External logging libraries like Winston are ideal for these scenarios.

External loggers provide more advanced features that go beyond the basics. If your application requires structured logging, integration with logging services, or advanced filtering, an external logger is likely the best choice.

Step-by-Step Guide

Step 1: Install the Winston Package

Start by installing Winston and the corresponding NestJS integration.

```
> _ Terminal Copy

npm install winston @nestjs/winston
```

Step 2: Configure Winston

Create a configuration for Winston that fits your needs.

```
> _ Index.js Copy

import * as winston from 'winston';
import { WinstonModule } from 'nest-winston';

const logger = WinstonModule.createLogger({
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.timestamp(),
        winston.format.printf(({ timestamp, level, message }) => {
          return `${timestamp} ${level} ${message}`;
        })
      ),
    },
  ],
});
```

Step 3: Use Winston in the Application

Pass the Winston logger instance when creating the NestJS application.

For example:

```
> _ Index.js Copy

const app = await NestFactory.create(AppModule, {
  logger,
});
await app.listen(3000);
```

Step 4: Inject Winston for Application Logging

Just like with the custom logger, you can inject the Winston logger into your services.

```
> _ Index.js Copy

import { Injectable } from '@nestjs/common';
import { Logger } from 'winston';
import { InjectLogger } from 'nest-winston';

@Injectable()
export class MyService {
  constructor(@InjectLogger() private readonly logger: Logger) {}

  doSomething() {
    this.logger.info('Doing something...');
  }
}
```

Conclusion

Setting up logging in NestJS can be as simple or as advanced as your application needs. Whether you're using the built-in logger for basic logging tasks, implementing a custom logger for more control, or integrating an external logger like Winston for advanced features, NestJS offers the flexibility to meet your needs.

By following the steps in this guide, you can ensure that your NestJS application is equipped with a robust logging system, tailored to your specific requirements.